Universidad Tecnológica Nacional

UTN

Metodología de Sistemas II

TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Solicitudes Académicas (SGSA)

Análisis de calidad, aplicación de refactorings y pruebas de regresión

Materia: Metodología de Sistemas II

Carrera: Tecnicatura Universitaria en Programación

Estudiante: Matías Gastón Farfán

Comisión: 7

Año: 2026

Índice

1. Introducción y descripción del sistema 3
2. Análisis de calidad del software 4
3. Diagnóstico de Code Smells y deuda técnica 5
4. Análisis de principios SOLID y Clean Code 6
5. Plan de Refactoring 7
6. Refactorings aplicados 8
7. Pruebas automatizadas de regresión 9
8. Code Review 10
9. Conclusiones finales 11
10. Repositorio, evidencias y uso de inteligencia artificial 12

1. Introducción y descripción del sistema

El Sistema de Gestión de Solicitudes Académicas (SGSA) es una aplicación de consola desarrollada en Java 21, utilizando IntelliJ IDEA y Gradle. Permite administrar solicitudes estudiantiles mediante el registro, consulta y seguimiento de trámites académicos.

El sistema permite registrar solicitudes, listarlas, buscarlas por identificador, filtrarlas por estado, asignar responsables, modificar estados y generar reportes. Sus entidades principales son Estudiante, Responsable y Solicitud, mientras que SistemaSolicitudes administra las operaciones.

La versión inicial utiliza una colección ArrayList para almacenar los datos temporalmente en memoria, sin base de datos ni autenticación. Su estructura original estaba compuesta por cinco clases Java.

Este proyecto se utilizó como punto de partida para evaluar la calidad del software, identificar code smells y deuda técnica, aplicar principios SOLID y Clean Code, realizar refactorings controlados y verificar los resultados mediante pruebas de regresión.

2. Análisis de calidad del software

Se evaluó la calidad externa e interna de la versión inicial del SGSA para identificar sus fortalezas y limitaciones antes de aplicar los refactorings.

Calidad externa: las pruebas manuales confirmaron el funcionamiento de las operaciones principales, como registrar, buscar y modificar solicitudes. El menú de consola facilita su utilización, aunque presenta limitaciones en la validación de entradas y no dispone de almacenamiento permanente.

Calidad interna: el sistema presenta nombres descriptivos y una estructura básica de clases. Sin embargo, se identificó una concentración de responsabilidades en Main y una combinación de lógica de gestión y presentación en SistemaSolicitudes, lo que dificulta su mantenimiento.

Deuda técnica: los principales aspectos detectados fueron los métodos extensos, las responsabilidades mezcladas, las validaciones incompletas y el uso de cadenas de texto para representar estados.

Este diagnóstico permitió establecer una base para aplicar mejoras de organización, legibilidad y mantenibilidad, conservando las funcionalidades existentes.

3. Diagnóstico de Code Smells y deuda técnica

El análisis del código original del SGSA permitió identificar ocho problemas de calidad que afectaban principalmente la organización, legibilidad y mantenibilidad del sistema.

Los problemas detectados fueron:

1. **Long Method:** método main() demasiado extenso.
2. **Múltiples responsabilidades:** concentración de tareas en Main.
3. **Mezcla de lógica y presentación:** uso de System.out.println() dentro de SistemaSolicitudes.
4. **Estados como cadenas:** utilización de String para representar estados.
5. **Validaciones incompletas:** falta de controles en determinadas entradas.
6. **Magic Numbers:** valores numéricos utilizados directamente en las opciones.
7. **Identificadores duplicados:** ausencia de controles al registrar solicitudes.
8. **Modificación directa del estado interno:** cambios de estado y responsable sin validaciones suficientes.

Los problemas se clasificaron según su riesgo e impacto, priorizando aquellos relacionados con la separación de responsabilidades, la validación de datos y la prevención de inconsistencias.

Este diagnóstico permitió fundamentar los refactorings posteriores y establecer las mejoras necesarias para aumentar la calidad interna del sistema.

4. Análisis de principios SOLID y Clean Code

Se evaluó el código original del SGSA considerando los principios SOLID y las buenas prácticas de Clean Code, con el propósito de identificar oportunidades de mejora.

Principios SOLID:

- **SRP (Responsabilidad Única):** cumplimiento parcial, debido a la concentración de tareas en Main y SistemaSolicitudes.
- **OCP (Abierto/Cerrado):** presenta limitaciones para incorporar nuevas funcionalidades sin modificar el código existente.
- **LSP (Sustitución de Liskov):** no aplicable directamente, ya que no existen jerarquías de herencia.
- **ISP (Segregación de Interfaces):** no evaluable directamente, porque el sistema no define interfaces.
- **DIP (Inversión de Dependencias):** presenta limitaciones por el acoplamiento entre la gestión de solicitudes y la presentación por consola.

Clean Code:

El sistema utiliza nombres descriptivos y una estructura comprensible. Sin embargo, se identificaron métodos extensos, responsabilidades mezcladas, validaciones incompletas y oportunidades de reutilización de código.

Conclusión: el análisis permitió fundamentar los refactorings orientados a mejorar la legibilidad, organización y mantenibilidad del sistema, sin incorporar complejidad innecesaria.

5. Plan de Refactoring

A partir del diagnóstico de code smells y del análisis SOLID y Clean Code, se planificaron cuatro refactorings para mejorar la calidad interna del SGSA.

Refactoring	Problema identificado	Mejora propuesta
1. Extract Method	Método main() extenso.	Dividir las operaciones en métodos más pequeños.
2. Move Method	Mezcla de gestión y presentación.	Trasladar la visualización a una clase específica.
3. Introduce Parameter Object	Constructor con múltiples parámetros.	Agrupar los datos mediante DatosSolicitud.
4. Replace String with Enum	Estados representados mediante cadenas.	Incorporar EstadoSolicitud para controlar los valores permitidos.

Estrategia de implementación: los cambios se planificaron de manera progresiva, con pruebas de regresión antes y después de los refactorings, evidencias comparativas y commits individuales en GitHub.

La versión original se conservó mediante el tag estado-inicial, permitiendo comparar ambas implementaciones.

6. Refactorings aplicados

Se implementaron cuatro refactorings controlados para mejorar la estructura interna del SGSA, manteniendo las funcionalidades existentes.

1. Extract Method: se reorganizó Main.java, extrayendo métodos para mostrar el menú, registrar, buscar y filtrar solicitudes, asignar responsables y cambiar estados. Esto mejoró la legibilidad del método principal.

2. Move Method: se creó PresentadorSolicitudes.java para trasladar la visualización detallada de solicitudes fuera de SistemaSolicitudes.java, logrando una separación parcial de responsabilidades.

3. Introduce Parameter Object: se incorporó DatosSolicitud.java para agrupar los parámetros necesarios al crear solicitudes. Se conservó el constructor original para mantener la compatibilidad.

4. Replace String with Enum: se creó EstadoSolicitud.java para definir los estados permitidos. También se incorporó una validación que rechaza estados incorrectos sin interrumpir la ejecución del programa.

Resultados: después de los cambios se ejecutaron las pruebas de regresión mediante Gradle, obteniendo 14 pruebas aprobadas, sin fallos y con resultado BUILD SUCCESSFUL.

Las comparaciones del código antes y después, junto con las capturas de verificación, se encuentran en docs/evidencias/refactoring-1/ hasta docs/evidencias/refactoring-4/.

7. Pruebas automatizadas de regresión

Para verificar que los refactorings conservaran el funcionamiento del SGSA, se implementaron **14 pruebas automatizadas con JUnit**, ejecutadas antes y después de las modificaciones.

Las pruebas evaluaron el registro, búsqueda, listado y filtrado de solicitudes, la asignación de responsables, los cambios de estado y la generación de reportes.

Comparación de resultados:

Indicador	**Antes**	**Después**
Pruebas ejecutadas	14	14
Pruebas aprobadas	14	14
Pruebas fallidas	0	0
Pruebas omitidas	0	0
Porcentaje de éxito	100 %	100 %
Resultado de Gradle	BUILD SUCCESSFUL	BUILD SUCCESSFUL

Las pruebas se ejecutaron mediante el comando .\gradlew.bat clean test, utilizando la misma batería de casos en ambas versiones.

Además, se realizó una prueba manual ingresando el estado no permitido cancelada. El sistema informó el error y continuó funcionando sin interrupciones.

Conclusión: los resultados demostraron que las funcionalidades cubiertas por las pruebas conservaron su comportamiento esperado después de los cuatro refactorings, sin detectar regresiones en los escenarios evaluados.

Las evidencias se encuentran en docs/evidencias/pruebas-antes/ y docs/evidencias/refactoring-4/.

8. Code Review

Se realizó una revisión del código del SGSA después de implementar los cuatro refactorings, analizando las ocho clases de producción para evaluar su organización, legibilidad y mantenibilidad.

Mejoras identificadas:

- Métodos más pequeños y organizados en Main.
- Separación parcial de la presentación mediante PresentadorSolicitudes.
- Agrupación de parámetros mediante DatosSolicitud.
- Validación de estados permitidos mediante EstadoSolicitud.

Deuda técnica pendiente:

La revisión identificó aspectos que todavía requieren mejoras, principalmente las validaciones incompletas de entradas, la falta de control de identificadores duplicados y la presencia de instrucciones de presentación dentro de SistemaSolicitudes.

También se recomendaron controles adicionales de datos, mayor centralización del manejo de estados y pequeñas mejoras de nomenclatura.

Conclusión: los refactorings mejoraron la calidad interna del sistema y las 14 pruebas automatizadas continuaron aprobándose. Las observaciones pendientes quedaron documentadas para futuras mejoras, sin modificar la versión validada y registrada con el tag estado-refactorizado.

9. Conclusiones finales

El Trabajo Práctico Integrador permitió analizar y mejorar la calidad interna del Sistema de Gestión de Solicitudes Académicas (SGSA), aplicando principios SOLID, buenas prácticas de Clean Code y cuatro técnicas de refactoring.

Las modificaciones realizadas mejoraron la organización, legibilidad y mantenibilidad del código. Mediante **14 pruebas automatizadas de regresión**, ejecutadas antes y después de los cambios, se verificó que las funcionalidades evaluadas conservaron su comportamiento esperado, con un **100 % de pruebas aprobadas**.

El uso de GitHub permitió documentar el proceso y conservar las versiones estado-inicial y estado-refactorizado para su comparación.

El Code Review también permitió identificar mejoras pendientes, principalmente en la validación de datos, el control de identificadores duplicados y la separación completa de responsabilidades.

En conclusión, el trabajo demostró la importancia de realizar mejoras progresivas, acompañadas de pruebas y control de versiones, para aumentar la calidad del software sin comprometer su funcionamiento.

10. Repositorio, evidencias y uso de inteligencia artificial

El desarrollo del Trabajo Práctico Integrador se documentó mediante GitHub, conservando el código fuente, los análisis técnicos, las pruebas automatizadas y las evidencias de los refactorings.

Repositorio: https://github.com/matfarfan/SGSA

Se utilizaron los tags estado-inicial y estado-refactorizado para identificar y comparar ambas versiones del sistema.

Diagramas UML: el diagrama inicial se encuentra en docs/evidencias/estado-inicial/uml-estado-inicial.md y el diagrama final en docs/evidencias/estado-refactorizado/SGSA-UML-Final.md. Ambos permiten comparar la estructura de clases antes y después de los refactorings.

Uso de inteligencia artificial: se utilizó ChatGPT como herramienta de apoyo para el análisis de calidad, la planificación de refactorings, la preparación de pruebas automatizadas y la revisión del código.

Su utilización quedó documentada mediante los prompts, las respuestas obtenidas y las capturas de las conversaciones, conservadas como evidencias complementarias del proyecto.